

Esercizio 1: [2 punti] Barrare tutte le frasi vere.

1. Nel primo Fortran l'occupazione di memoria di un programma era nota a compile time. [X]
2. HTTP (senza scripts) è un linguaggio general purpose. []

Esercizio 1: [2 punti] Barrare tutte le frasi vere.

1. Se in Java si usa unicamente il polimorfismo parametrico allora tutti controlli di tipo avvengono a tempo di compilazione. [X]
2. Nel paradigma funzionale puro non ci sono gli assegnamenti. [X]

Esercizio 1: [2 punti] Barrare tutte le frasi vere.

1. Il C++ ha un garbage collector. []
2. Il primo Fortran supportava una operazione analoga a una malloc. []

Esercizio 1: [4 punti] Barrare tutte le frasi vere.

1. La JVM può caricare classi da host diversi [X]
2. Un linguaggio puramente funzionale ha i cicli while []
3. La promozione automatica dei tipi numerici è una forma di polimorfismo ad hoc [X]
4. Il C adotta sempre la structural equivalence tra tipi []

Esercizio 1: [4 punti] Barrare tutte le frasi vere.

1. Il predicato Prolog member può enumerare i membri di una lista e generare liste [X]
2. Tra i compiti del supporto a run time c'è la gestione della memoria [X]
3. I linguaggi funzionali non sono computazionalmente completi []
4. In C++ l'ereditarietà multipla può causare un consumo esponenziale di memoria [X]

Esercizio 1: [4 punti] Barrare tutte le frasi vere.

1. La proprietà di invertibilità dei linguaggi logici prende il nome dal fatto che un unico programma può essere usato per calcolare sia una funzione sia la sua inversa [X]
2. Il primo Fortran aveva uno stack di attivazione []
3. Il linguaggio SQL è computazionalmente completo []
4. In C++ gli interi (int) sono lunghi 32 bit []

Esercizio 1: [2 punti] Barrare tutte le frasi vere.

1. In Java, se l'unico polimorfismo universale usato è quello per inclusione, allora tutti gli eventuali errori di tipo sono segnalati a tempo di compilazione []
2. Il primo Fortran supportava un analogo della malloc []
3. Il Prolog puro supporta i cicli for/next []
4. Java non ha metodi analoghi alla funzione free() di C e C++ [X]

Esercizio 1: [2 punti] Barrare tutte le frasi vere.

1. Il primo Fortran supporta la ricorsione []
2. Il LISP supporta la ricorsione [X]
3. Java ha una implementazione: compilata [] interpretata [] mista [X]
4. Esistono linguaggi a oggetti senza gerarchia di tipi []

Esercizio 1: [2 punti] Barrare tutte le risposte corrette.

Dire se il polimorfismo parametrico puro (quello dei template):

1. E' più adatto del polimorfismo per inclusione per rappresentare insiemi di oggetti omogenei SI [X] NO []
2. E' più adatto del polimorfismo per inclusione per rappresentare insiemi di oggetti eterogenei SI [] NO [X]
3. Ammette il controllo di tipi completo a tempo di compilazione SI [X] NO []
4. Può generare errori di tipo a runtime SI [] NO [X]

Esercizio 1: [2 punti] Barrare tutte le frasi vere.

- ☒ Il controllo di correttezza dei downcast richiede controlli a runtime.
- ☐ Se A è una istanza di B e B è una istanza di C allora A è una istanza di C.
- ☒ Se A è una istanza di B e B è una sottoclasse di C allora A è una istanza di C.
- ☐ La funzione *membro(elemento,lista)* nel paradigma funzionale può essere usata anche per generare tutti gli elementi della lista.
- ☐ Le macro (come ad es. le *#define* del C) hanno un proprio ambiente locale implementato con un record di attivazione.

Esercizio 1: [2 punti] Barrare tutte le frasi vere.

- ☐ Il primo Fortran supportava la ricorsione. No, gestiva la memoria staticamente, niente stack!
- ☒ Se il linguaggio è dinamicamente tipato, allora il tipo di una variabile può cambiare durante l'esecuzione del programma. Come negli scripting languages, per esempio
- ☐ C adotta la structural equivalence per le `struct` e la name equivalence per gli altri tipi. Il contrario!
- ☒ Si può accedere alle variabili non locali di una procedura in tempo costante, indipendentemente da quanti record di attivazione si devono attraversare. Vedere l'es. 2

Esercizio 1: [2 punti] Barrare tutte le frasi vere.

- a) ☐ I linguaggi funzionali puri non sono computazionalmente completi
- b) ☐ Compilatore e programmi oggetto sono di norma eseguiti dalla stessa macchina
- c) ☒ C è debolmente tipato
- d) ☐ Nei linguaggi fortemente tipati il type checking avviene tutto a tempo di compilazione